RELAY: Central Workflow & Working MVP (Round 2 — The Build)

Project: Relay (Evidence-Linked Knowledge Handover Platform)
Team Name: SoloStack (1-Member Solo Track)
Author: Anshika Aggarwal | B.Tech 1st Year, Dr. Akhilesh Das Gupta Institute of Professional Studies (ADGIPS), New Delhi

Stage: Round 2 — The Build Submission
Target Criteria: Core Workflow Runs End-to-End (30 Pts)
Evaluation Mode: Autonomous Judge Quick-Start

EXECUTIVE BUILD STATEMENT — ROUND 2 DELIVERABLES

For Round 2 (The Build), **Team SoloStack** has engineered a fully functioning, end-to-end knowledge handover platform. The central workflow runs seamlessly from raw source ingestion to role-tailored onboarding without requiring any external walkthrough. Judges can interact directly using the built-in **'1-Click Benchmark Quick-Start'** or by connecting custom Git logs and notes. The core application features automated pre-ingestion secret sanitization, bi-directional evidence citations, an algorithmic contradiction radar (detecting discrepancies between task boards and commit activity), and an interactive dual-pane review canvas.

1. Working Project Link & Source-Code Repository

To guarantee effortless evaluation by the judging panel without setup friction, Relay is available through two deployment modes:

Deliverable	URL / Access Path	Evaluation Notes for Judges
Source-Code Repository	https://github.com/AnshikaAggarwal23/relay-knowlegde-handover *(Public GitHub Repository)*	Complete source code including FastAPI backend, React client, automated tests, and preloaded benchmark dataset.
Working Project Link (Live Web App)	https://anshikaaggarwal23.github.io/relay-knowlegde-handover/ *(Live Cloud Deployment via GitHub Pages)*	Zero-install live web client. Features built-in autonomous mode and 1-Click Judge Quick-Start.
Local 1-Command Runner	python backend/app/main.py *(Runs at http://localhost:8000)*	Unified server: FastAPI serves the interactive web client and OpenAPI Swagger documentation at /docs.

2. The Central Workflow: Autonomous Judge Journey (Judging Criterion: 30 Pts)

A judge can launch and evaluate the entire transition pipeline from start to finish without requiring any manual assistance:

Step	Workflow Stage	Judge Action & What Happens On Screen
Step 1	**Connect & Ingest**	Judge opens the app and clicks **'■ Judge Quick-Start'**. The system loads 6 authorized project sources (Git commits, PRs, task cards, Discord notes). The **Secret Redaction Radar** immediately flashes green, showing 2 sensitive credentials (an API key and a server password) scrubbed from the view.
Step 2	**Reconstruct State**	Judge clicks 'Proceed to Stage 2: Reconstruct'. The backend processes the telemetry, categorizing findings into Architectural Decisions, Fragile Gotchas, Ownership, and Next Actions.
Step 3	**Verify & Conflict Radar**	Judge reviews claims with confidence ratings. Relay automatically triggers an amber **Contradiction Alert**: Task card TASK-42 claimed 'Done' on Oct 24, but Git commit #9b2c3de rolled it back on Oct 26 due to Safari iOS issues. Relay highlights this conflict rather than inventing false certainty.

Step 4	**Dual-Pane Review Canvas**	The core innovation: Left pane displays raw source evidence; right pane displays editable handover cards. Clicking any card highlights the exact sentence in the source on the left. The judge can edit text, add senior tips, and click **'Approve & Seal Handover'**.
Step 5	**Role-Tailored Handover**	The sealed handover generates an actionable onboarding view. Judge toggles between **Developer View** (interactive Day-1 checklist, gotchas, SQLite setup) and **Lead View** (milestone risks, stakeholder contacts). Features 1-click Export to Markdown / PDF.

3. List of Completed Features (Working in Round 2)

• **1-Click Judge Quick-Start Benchmark Engine:** Loads a realistic 4-month transition dataset from 'FestFlow' (Campus Fest Web Portal) so evaluators can test immediately without hunting for files.

• **Pre-Ingestion Secret & PII Sanitizer:** Regex-based protection pipeline that intercepts and redacts API keys, Bearer tokens, GitHub PATs, and passwords before processing.

• **Structured Entity Extraction Engine:** FastAPI backend endpoint that structures unstructured Git commit messages, PRs, and chats into decisions, blockers, ownership, and next steps.

• **Cross-Source Temporal Contradiction Radar:** Rule pass comparing timestamps across task boards and Git commit logs to flag discrepancies (e.g. task marked closed vs commit reverted).

• **Dual-Pane Interactive Review Canvas:** Side-by-side desktop UI: Raw Evidence Inspector on the left, editable Handover Cards on the right with bi-directional highlight synchronization.

• **Role-Tailored Output Generator:** Interactive view switcher providing tailored perspectives: Developer View (interactive checklist, gotchas) vs. Project Lead View (risks, contacts).

• **Immutable Handover Manifest & Export:** Generates timestamped handover seal with one-click Markdown summary export and printer-friendly PDF dossier generation.

4. Technology and Implementation Details (Depth & Reliability: 25 Pts)

Relay's implementation emphasizes technical reliability, clean separation of concerns, and zero external dependency failures:

Component	Technology & Implementation Architecture
Backend API Architecture	Built with **FastAPI (Python 3.11+)** and **Pydantic v2**. Asynchronous REST endpoints handle source parsing, secret sanitization, claim updates, and handover sealing. Automatic interactive Swagger documentation is provided at /docs.
Frontend Client Architecture	Built with **React 18** and **Tailwind CSS**. Implements a responsive 5-stage stepper, dual-pane review canvas with bi-directional citation inspection, and interactive task checklist state management. Features an autonomous browser fallback so judges can evaluate the UI even without the local Python server running.
AI / Extraction Strategy	Configured for **Google Gemini 1.5 Flash** with strict JSON schema enforcement. Crucially, Relay incorporates a deterministic semantic analyzer fallback that guarantees instantaneous, bug-free execution during live hackathon judging, completely immune to rate limits or API key outages.
Security & Sanitizer Engine	Custom regex compilation rules targeting GitHub PATs (ghp_*), AWS access keys, Bearer tokens, and database URI connection strings, replacing them with formatted safe tokens prior to display.

5. Explanation of Changes Made to the Original Plan (Progress: 20 Pts)

During Round 2 implementation, Team SoloStack made three justified architectural improvements to enhance reliability and judge usability:

• **Addition of 1-Click Judge Quick-Start Benchmark:** In the original Round 1 plan, users were expected to upload custom files manually. In Round 2, we added a pre-configured benchmark project ('FestFlow Campus Portal') with realistic Git commits and chat notes, allowing judges to test the complete workflow in a single click.

• **Dual-Mode Execution (Autonomous + Backend Connected):** Rather than requiring judges to configure Python virtual environments and node dependencies, the React frontend operates in a dual mode: it seamlessly queries the live FastAPI backend when available, but automatically runs its built-in state engine if evaluated as a standalone web client.

• **Elevated Temporal Contradiction Flagging:** Rather than treating contradictions as a passive status tag, we built a dedicated high-visibility **Contradiction Radar Banner** in Stage 3 to immediately alert the evaluator when task boards disagree with Git commit history.

6. Three Substantial Improvements Committed for Round 3 (15 Pts)

In strict compliance with hackathon guidelines requiring meaningful product improvements rather than cosmetic tweaks, Team SoloStack commits to delivering the following three substantial features in Round 3:

Commitment	Feature Name	Substantial Technical Scope & Impact in Round 3
Improvement 1	**Cross-Source Temporal Timeline Slider**	An interactive chronological scrub bar that displays when decisions were made across all sources on a synchronized timeline. Allows the reviewer to visually trace how a feature moved from an initial Discord chat to a Jira ticket, into a Git commit, and finally resolved or reverted.
Improvement 2	**Cryptographic Handover Seal & Tamper-Proof Audit Manifest**	Generates an immutable SHA-256 hash manifest upon handover approval. Embeds verification metadata into an encrypted, downloadable handover dossier so future maintainers can cryptographically verify that handover notes were approved by the departing lead.
Improvement 3	**Interactive Handover Q&A; Agent with Strict Citation Guardrails**	An embedded conversational assistant for incoming developers that answers questions about the project (e.g., 'How do I run the ticket generator?'). Strictly constrained via RAG to cite only verified claims from the sealed handover manifest, refusing to hallucinate.

7. Complete Participant Details & Submission Integrity

Field	Official Record
Team Name	**SoloStack** (1-Member Solo Participant Track)
Participant Name	Anshika Aggarwal
Academic Standing	B.Tech 1st Year (Passionate Aspiring Full-Stack Developer)
Institution	Dr. Akhilesh Das Gupta Institute of Professional Studies (ADGIPS), GGSIPU, New Delhi
Designated Role	Solo Full-Stack Developer, Product Architect & System Integrator
Build Integrity	All code developed during the official Round 2 build window in full compliance with HackWithPune fair participation rules.